Polynomial-Delay Enumeration of All Potential Maximal Cliques

ZIJIAN ZENG, UCSI University, Malaysia

A potential maximal clique of a graph is a maximal clique in an inclusion-minimal chordal completion. We give a deterministic algorithm enumerating all nonempty potential maximal cliques of every finite simple undirected graph exactly once, with polynomial delay before the first output, between consecutive outputs, and from the last output to termination. The algorithm allows exponential storage and imposes no bound on output size.

The construction builds on the recursive seed and neighborhood framework of Bergé, Limouzy and Schivre. During minimal-separator enumeration, it emits helper potential maximal cliques whose preimage sets have size at most the number of vertices. These emissions make separator prefixes chargeable to actual outputs. Exact suspension at recursive outputs, together with an injective lift, permits a simultaneous induction on output-rank time and termination time with an additive polynomial recurrence. Standard buffering then supplies worst-case delay, including both boundary waits. We specify the graph-search neighborhood explicitly and verify that it retains the structural predecessors used in the source framework.

Author's Contact Information: Zijian Zeng, UCSI University, Kuala Lumpur, Malaysia, 1002266693@ucsiuniversity.edu.my.

1 The guarantee and its context

Let $G = (V, E)$ be a finite simple undirected graph, with $n = |V|$ and $m = |E|$. A *triangulation* is a chordal supergraph on the same vertex set; it is *minimal* if no proper intermediate supergraph is chordal. Minimality is by inclusion, rather than by the number of added edges. A nonempty set is a *potential maximal clique* (PMC) if it is a maximal clique of some minimal triangulation. Write $\mathcal{P}(G)$ for this family.

Theorem 1.1. *There is one deterministic algorithm and one fixed polynomial P such that, on every graph G, the algorithm enumerates $\mathcal{P}(G)$ without repetition. Time before the first output, between consecutive outputs, and after the last output until termination is at most $P(n + m + 1)$. If $V = \emptyset$, it terminates within the same bound without output. Exponential space is allowed.*

PMCs provide an enumeration interface for exact graph algorithms, notably algorithms based on minimal triangulations and treewidth. Bergé, Limouzy and Schivre [BLS26], abbreviated BLS below, give total time $O(n^4 m|\mathcal{P}(G)|)$ in Theorem 1 and explicitly leave polynomial delay as an open challenge in Section 1.4. A total-time bound of this form allows a long wait early in the computation or after the final output; it does not by itself prove Theorem 1.1.

The nearest delay guarantees have different scopes. Grigoriev, Kobayashi, Tamaki and van der Zanden [GKTvdZ25], Theorem 32, give polynomial delay for triconnected plane graphs. Kenig [Ken26], Theorem 1.1, gives delay $2^{O(k^3)} n^4 \log n$ for PMCs of size at most k, including prescribed and excluded vertices; setting $k = n$ does not give input-polynomial delay. Brosse, Conte, Limouzy, Punzi and Rucci [BCL+24], Lemma 4, obtain output-polynomial time with polynomial space. The present algorithm covers all graphs and all sizes but does not establish polynomial space, an improved treewidth running-time exponent, or the constrained-search guarantee of [Ken26].

Our contribution is a streaming modification and timing analysis of the BLS framework. Recursive outputs are processed immediately, and a minimal separator produces a valid helper PMC before separator enumeration resumes. A witness bounds each helper fiber by n, so an arbitrarily long separator prefix requires proportionally many already generated solutions. We prove both a linear bound in every output rank and a linear total bound, with polynomial input factors. The second invariant pays for recursive finalization and the final delay. The recurrence adds local costs instead of multiplying the previous dimension's coefficient.

PMC recognition, recursive lifting and structural predecessor existence are inherited from BLS and earlier work. Separator extension already appears in [GKTvdZ25, Proposition 29]; the output-count inequality between separators and PMCs is not claimed here as an innovation. Conversion from polynomial amortized delay to worst-case delay with a potentially exponential FIFO is standard; see Capelli and Strozecki [CS23, CS26]. The additional guarantee comes from proving the requisite prefix bound for this PMC generator, rather than applying buffering to an aggregate total-time bound.

2 Conventions and established primitives

For $X \subseteq V$, write $G - X$ for vertex deletion and $N(X)$ for the open neighborhood outside X. When $v \notin X$, let $C_G(v, X)$ be the component of $G - X$ containing v. A component C of $G - S$ is *full* for S

if $N(C) = S$. A set S is a minimal separator precisely when it has at least two full components. Let $\mathcal{D}(G)$ be the set of minimal separators.

We use the following standard facts, with theorem numbers referring to the specified version of BLS [BLS26].

(1) A nonempty Ω is a PMC if and only if no component of $G - \Omega$ has neighborhood Ω and every nonadjacent pair of vertices in Ω belongs to the neighborhood of some component of $G - \Omega$ (Theorem 4 and Lemma 7). This is a polynomial recognition test.
(2) For a vertex a and $X \in \mathcal{P}(G-a)$, at least one of X and $X \cup \{a\}$ belongs to $\mathcal{P}(G)$ (Lemma 11). In fact the extension is unique by [BCL+24, Proposition 1]; only existence is needed here. Every PMC containing a is $X \cup \{a\}$ for a child PMC X, or $S \cup \{a\}$ for $S \in \mathcal{D}(G)$ (Lemma 12).
(3) Minimal separators admit polynomial-delay enumeration, as stated in BLS Lemma 1 from Berry, Bordat and Cogis [BBC99].
(4) A minimal triangulation can be computed in polynomial time by MCS-M of Berry, Blair, Heggernes and Peyton [BBHP04], Theorem 4.4.

For clarity about endpoints in the separator primitive, one may implement the known component-neighborhood scheme with a queue and a trie. Initialize with $N(C)$ for all components C of $G-N[v]$, for every vertex v. On dequeuing S, emit it; then, for every $x \in S$, insert $N(C)$ for components C of $G - (S \cup N(x))$. Insert only unseen sets and, if desired, validate the two-full-components criterion. The known completeness theorem supplies all separators. Initialization and each queue expansion require polynomially many graph searches. Emitting *before* expansion gives polynomial initial, successive and final delay, including the empty-family case. No exhaustive subset oracle is part of this primitive.

For MCS-M, the path tests in its primary pseudocode can be performed naively by graph search in the fixed input block. All tests for a numbering step use the weights before that step's increments. There are polynomially many tests, so no optimized implementation is required here. A perfect elimination order of the resulting chordal graph lists its maximal cliques in polynomial time: take each vertex with its later neighbors and discard nonmaximal candidates.

Sets are n-bit vectors. Use binary tries of depth at most n for dictionaries and linked queues, so no operation scans all previous outputs. Graph searches, recognition and serialization have polynomial worst-case cost. Store at most exponentially many records at each of at most n recursive levels; pointers and counters therefore have polynomial bit length. We use a pointer RAM with polynomial-length words, charging polynomial bit cost to its operations. Memory may be retained until termination: an exponential deallocation pass must not be appended after the last output.

3 A separator helper with a witnessed fiber bound

Lemma 3.1. *For a connected graph G, a deterministic polynomial-time map $\Gamma : \mathcal{D}(G) \to \mathcal{P}(G)$ can be chosen so that every image has at most n preimages. More precisely, for each S, some component D of $G - \Gamma(S)$ satisfies $N_G(D) = S$.*

Proof. Choose two full components C, D of $G - S$ using fixed label order. For every component C_i of $G - S$, make S a clique in $G[S \cup C_i]$ and compute a minimal triangulation T_i of this saturated block. Their union T is chordal: its pieces are chordal, intersect in the clique S, and have no edges between different C_i.

We check minimality with respect to the original graph. Let H be chordal with $G \subseteq H \subseteq T$. If an edge xy inside S is absent from H, fullness and connectivity give an x–y path with interior in C and another with interior in D. Choose shortest such paths in H on their respective sides. Both are induced and have length at least two. Their interiors have no cross edges in T and hence none in H, so their union is a chordless cycle of length at least four. This contradicts chordality. Thus S is a clique in H. Each $H[S \cup C_i]$ is now a chordal supergraph of the saturated block contained in T_i, and equals T_i by minimality. Consequently $H = T$, proving that T is a minimal triangulation of G.

There is a maximal clique of T_C properly containing S. To justify this extension, take a clique tree of the connected chordal graph T_C. For each $s \in S$, the nodes whose cliques contain s form a subtree; these subtrees pairwise intersect because S is a clique. The union of nodes whose cliques meet C is also a connected subtree: the subtrees for vertices of connected C are connected through its edges. It meets each subtree for $s \in S$ because C is full. The Helly property for subtrees of a tree gives a node containing all of S and a vertex of C. For completeness, tree Helly follows by rooting the tree and choosing the deepest of the nearest-to-root vertices of the pairwise intersecting subtrees; that vertex lies in every subtree.

Choose deterministically a maximal clique Γ of T_C containing S properly. It is maximal in T too, since no other block contains a vertex adjacent to its vertex in C. Thus $\Gamma \in \mathcal{P}(G)$. We have $S \subsetneq \Gamma \subseteq S \cup C$, so D survives as a whole component of $G - \Gamma$ and $N_G(D) = S$. For a fixed image Γ, distinct preimage separators are distinct neighborhoods of components of $G - \Gamma$. There are at most n such components. All choices and computations above take polynomial time. □

The useful assertion is about the *actual map*: after processing any r distinct separators, at least r/n distinct helper PMCs have appeared. They may coincide with earlier outputs of another phase; those earlier outputs still pay for their separator preimages. A final inequality $|\mathcal{D}(G)| \leq n|\mathcal{P}(G)|$ alone would not account for a prefix if the separator phase did not actually emit these helpers.

4 An explicit neighborhood and the inherited preimages

Call a PMC Ω *a-nice* if $a \notin \Omega$ and $\Omega \notin \mathcal{P}(G - a)$, following BLS Definition 16. Non-a-nice PMCs are the seed family. BLS Theorems 8–10 supply predecessors for a-nice targets. We state the portion of their structural construction used here, and then connect it to an explicit executable neighborhood.

Proposition 4.1 (Structural witnesses from BLS). *Let Ω be a-nice, let $C_a = C_G(a, \Omega)$, $S_0 = N(C_a)$ and $T_0 = \Omega \setminus S_0$. Then T_0 is nonempty and lies in a full component $C_T \neq C_a$ of $G - S_0$. There are $W \in \mathcal{P}(G)$ and $u \in W \cap C_a$ with $T_0 \subseteq W$ and*

$$\Omega \subseteq X := W \cup N(u) \subseteq \Omega \cup C_a. \tag{1}$$

Moreover, either $a \in W$, or $a \notin W$ and $C_G(a, W) \subsetneq C_a$.

These are inherited structural facts, rather than a new characterization. The placement of T_0 is BLS Lemma 10. To make the provenance of (1) explicit, Theorem 9 constructs $W = \{a\} \cup \widehat{S} \cup T_0$ with $u = a$, and every missing vertex of S_0 is adjacent to a. Theorem 10 constructs $W = D \cup (\Omega \setminus S_D) \cup S'_D$, where $D \subseteq C_a$, $S'_D \subseteq S_D$, and every missing vertex of S_D is adjacent to the selected $u \in D$. These statements give both inclusions in (1) and retain T_0. The latter theorem also gives the strict decrease of the a-component. Existence of the requisite alternatives is supplied by Theorem 8.

Here is the neighborhood actually computed. For $W \in \mathcal{P}(G)$ and each ordered pair of distinct $u, b \in W$:

(1) Put $X = W \cup N(u)$ and let $A = C_G(b, X \setminus \{b\})$.
(2) Reject if $u \in A \cup N(A)$. Otherwise let $U = C_G(u, N(A))$ and $S = N(U)$.
(3) Let $B = C_G(b, S)$ and $Z = X \cap (S \cup B)$. Retain Z exactly when it is a nonempty PMC.

All component queries are defined. In particular, $b \in A$ and $S \subseteq N(A)$, so $b \notin S$; the guard ensures $u \notin N(A)$. There are at most n^2 pairs and polynomial work per pair.

BLS Definition 12 assumes that $X \setminus \{a\}$ is an (a, b)-separator, whereas Definition 20 chooses both endpoints in X. We avoid relying on this endpoint wording by defining the graph searches above. The following argument shows directly that they keep every structural arc needed for reachability.

Lemma 4.2. *For each target and witness in Proposition 4.1, every choice $b \in T_0$ makes the pair (u, b) survive the guard and return $Z = \Omega$. Consequently all PMCs are reachable from the non-a-nice PMCs by this neighborhood.*

Proof. Since $S_0 \subseteq X \setminus \{b\}$, the component A stays in C_T. It has no vertex or neighbor in C_a, a different component of $G - S_0$. For any $s \in S_0$, the PMC recognition criterion supplies either the edge bs or a b–s path with interior in one component of $G - \Omega$. In the latter case the interior lies in C_T, since the path cannot leave that component of $G - S_0$ before reaching s. It therefore avoids X by (1). Starting at b, the search defining A reaches a neighbor of s. Thus

$$S_0 \subseteq N(A), \qquad N(A) \cap C_a = \varnothing.$$

The guard keeps u. Removing $N(A)$ isolates the connected set C_a from everything else because it removes all of $S_0 = N(C_a)$ and none of C_a. Hence $U = C_a$ and $S = S_0$. Now $B = C_T$, and (1) gives $X \cap (S_0 \cup C_T) = \Omega$. Recognition accepts it.

Follow structural predecessors backwards from an a-nice PMC. Until a non-a-nice predecessor is reached, its nonempty a-component strictly decreases. This process cannot continue indefinitely and takes at most n steps. Reverse the sequence and use the proved computational arcs. □

5 The streaming generator

For now assume that H is connected and nonempty. Define a recursive generator $\mathcal{A}(H)$ which may suspend at an emission and resume from exactly that point. If H is complete, emit $V(H)$ and halt. Otherwise choose a so that $H - a$ is connected, for example a leaf of a fixed spanning tree.

Maintain an empty trie Seen and an empty linked queue L. The operation Insert(Z) checks the trie; if Z is unseen, it stores Z, appends Z to L and immediately emits Z, suspending before further work. All calls below pass recognized PMCs.

I. Run $\mathcal{A}(H - a)$ up to its next emission X and suspend the child. Test X and $X \cup \{a\}$ in fixed order for membership in $\mathcal{P}(H)$, inserting each valid set. Resume the child only after these tests and their emissions are finished. Repeat until the child halts.

II. Run the polynomial-delay minimal-separator enumerator on H. Suspend it at each distinct separator S. Compute and insert $\Gamma(S)$ from Lemma 3.1; then test $S \cup \{a\}$ and insert it if valid. Resume separator enumeration afterwards, continuing to its halt.

III. While L is nonempty, pop its first entry W, compute the neighbors of Section 4 in fixed order, and insert each valid neighbor. Halt when the queue is empty.

The queue is filled at discovery, including in phases I and II. No completed exponential seed list is copied at a phase transition.

Lemma 5.1. *The generator terminates and emits exactly $\mathcal{P}(H)$ without repetition. Different child PMCs have different parent lifts, so $|\mathcal{P}(H-a)| \le |\mathcal{P}(H)|$.*

Proof. Recognition and the trie give validity and uniqueness. Each child PMC has at least one valid lift, and deleting a from a lift recovers the child set. Thus distinct children supply distinct parent outputs. Phase I includes all parent PMCs that are PMCs of $H-a$. Every PMC containing a has one of the forms supplied in Section 2, and is found in phase I or II. These are exactly the non-a-nice PMCs. Extra helper seeds do not reduce reachability, so Lemma 4.2 proves completeness of phase III. At most $2^{|V(H)|}$ sets are inserted, each queue entry is popped once, and all phases consist of finite operations; recursion reduces the vertex count. Termination follows. □

6 One induction for output ranks and termination

Let $K = |\mathcal{P}(H)| \ge 1$, $K_c = |\mathcal{P}(H-a)|$ and $D = |\mathcal{D}(H)|$. We measure generator work, rather than time while a coroutine is externally suspended. Choose a fixed nondecreasing polynomial $p(n) \ge 1$ bounding each of: initialization and a complete-graph case; local processing of one child output; one separator gap together with its helper and lift processing; one entire queue expansion; and each of the boundedly many phase transitions and finalization operations. Include dictionaries, queues, serialization, coroutine bookkeeping and their polynomial bit costs. Such a computable polynomial exists by the specified implementations. Fix the implementation once, so its constants are independent of the input and output counts. Take $Q(1)$ large enough and set

$$Q(n) = Q(n-1) + (n+10)p(n) \quad (n \ge 2). \tag{2}$$

For instance $Q(1) + (n+1)(n+10)p(n)$ is a fixed polynomial upper bound.

Lemma 6.1. *For every connected nonempty n-vertex graph, simultaneously:*

$$E_H(j) \le Q(n)j \qquad (1 \le j \le K), \tag{3}$$

$$T_H \le Q(n)K, \tag{4}$$

where $E_H(j)$ is the work through the j-th distinct emission and T_H is the work through complete termination.

Proof. Induct on n, proving both assertions together. Complete graphs obey the bounds by the choice of p and Q. For a noncomplete graph, Lemma 5.1 gives $K_c \le K$, Lemma 3.1 gives $D \le nK$, and at most K queue entries are expanded. By the child's termination bound, its total cost is at most $Q(n-1)K_c$. All local work is at most

$$p(n)(K_c + D + K + 5) \le (n+7)p(n)K.$$

Together with (2) this proves (4). In particular the recursive coefficient is used once, not multiplied by an additional constant greater than one.

For (3), classify the j-th emission by its phase.

Phase I. Suppose it occurs while processing child emission r. The first $r-1$ child outputs already supplied distinct parent lifts, so $r \le j$. Because the child is suspended at emission r, its elapsed work is at most $Q(n-1)r$ by the child prefix bound. Local work so far is at most $p(n)(r+2) \le 3p(n)j$, proving the desired estimate. An emission during the first lift test does not resume the child before the second test.

Phase II. The child has halted and all its lifts are present, so $K_c \le j$. If the current separator has rank r, its $r-1$ predecessors have all had their helper PMCs emitted earlier, either in this phase or already in phase I. Each image has at most n preimages, hence

$$r-1 \le n(j-1), \qquad r \le nj.$$

The child cost is at most $Q(n-1)K_c$ by its termination bound. Local cost is at most $p(n)(K_c+r+4) \le (n+5)p(n)j$. These estimates prove (3), also when the current output is the separator lift rather than a newly emitted helper.

Phase III. Every separator helper has been emitted before this phase, so $D \le n(j-1)$. If q queue expansions have begun by this emission, each popped set was emitted earlier and is popped once; thus $q \le j$. Also $K_c \le j$. Child and local costs are at most

$$Q(n-1)K_c + p(n)(K_c + D + q + 5) \le [Q(n-1) + (n+7)p(n)]\,j.$$

This is bounded by $Q(n)j$. The simultaneous induction is complete. □

The distinction between the two invariants is essential. A recursive prefix bound alone does not control the child's final tail before phase II. The termination bound pays for that tail with the child's already lifted outputs. Conversely a total-time bound alone would not control the number of processed separators before the next fresh output. Their actual helper emissions provide that control.

7 Buffering and all input boundaries

We give the standard FIFO conversion explicitly to include termination. For a connected nonempty graph, use an integer polynomial upper bound $q \ge Q(n)$. Simulate $\mathcal{A}(H)$ in blocks of q steps, stopping a block early if the generator halts, and put its emissions in a FIFO buffer. After each block return one buffered solution. Once simulation has halted, drain the buffer with polynomial serialization time per solution and halt. Account for the polynomial enqueue cost in an emission's simulated cost; step counting and simulation overhead change the polynomial bound only by a polynomial factor.

At the end of block $j \le K$, the prefix bound ensures at least j generated solutions, whereas only $j-1$ have been returned. Thus the buffer is nonempty. If $j > K$, the termination bound says the generator has already halted by $qK < qj$, so no empty-buffer block can occur with it still running. If it halts within a block, all unreturned solutions are already buffered. It halts no later than the end of block K. Consequently after the K-th returned solution there is no remaining simulation tail. The first wait, each intermediate wait, and the final wait are polynomial. This is the usual amortized-to-delay buffering principle [CS23, CS26], applied to the two bounds just proved; it makes no polynomial-space assertion.

It remains to include disconnected and empty graphs. In a minimal triangulation of a disconnected graph there are no edges between original connected components: deleting all such

edges leaves the disjoint union of chordal induced subgraphs containing the original graph, so minimality forbids their presence. Conversely, a disjoint union of minimal triangulations of its components is minimal, since a proper chordal subtriangulation would restrict to a proper one on some component. Thus the nonempty PMCs of G are exactly the PMCs of its nonempty connected components, on their original labels.

Find these components in polynomial time. The zero-vertex graph halts without output. Otherwise run the buffered connected algorithm on the components sequentially. Each has an output, and nonempty sets from different components are distinct. At a component boundary, the previous final wait plus the next initial wait is polynomial in $n + m + 1$. Isolated vertices and complete components are included in the base case. This proves Theorem 1.1.

8 Verification and scope

The proof uses established structural theorems as specified above; finite experiments are supplementary diagnostics. A retained test record compares complete PMC and separator families on 29 specified graphs of order at most ten, with 471 PMC outputs in total, against exhaustive finite oracles. It also checks helper witnesses, fiber sizes, lifts, queue counts and absence of repeated outputs. A separate automated check compares the literal minimal-triangulation definition with the PMC criterion on all 1100 labelled graphs through order five, including the empty graph. It also independently checks the 29 serialized output families using that criterion. These checks do not prove the universal theorem or its asymptotic timing bounds. Exhaustive subsets are used by the finite verification oracles and are absent from the theoretical enumeration algorithm.

The theorem guarantees progress for every requested output prefix. It does not bound the number of PMCs polynomially, promise polynomial memory, rank outputs, or provide the inclusion/exclusion oracle of [Ken26]. The polynomial exponent is not optimized. Practical implementations and a polynomial-space delay algorithm require further work. The literature comparison concerns the cited primary versions; it does not certify exhaustive priority.

Generative AI materially assisted development of the proof and computational checks, as detailed in the statement at the end. Neither a model review nor a successful finite check is a formal proof certificate.

References

[BBC99] Anne Berry, Jean-Paul Bordat, and Olivier Cogis. Generating all the minimal separators of a graph. In *Graph-Theoretic Concepts in Computer Science (WG 1999)*, volume 1665 of *Lecture Notes in Computer Science*, pages 167–172. Springer, 1999. https://doi.org/10.1007/3-540-46784-X_17.

[BBHP04] Anne Berry, Jean R. S. Blair, Pinar Heggernes, and Barry W. Peyton. Maximum cardinality search for computing minimal triangulations of graphs. *Algorithmica*, 39:287–298, 2004. https://doi.org/10.1007/s00453-004-1084-3.

[BCL+24] Caroline Brosse, Alessio Conte, Vincent Limouzy, Giulia Punzi, and Davide Rucci. Output-sensitive enumeration of potential maximal cliques in polynomial space. arXiv:2402.18265v1; IWOCA 2024, 2024. https://arxiv.org/abs/2402.18265v1.

[BLS26] Pierre Bergé, Vincent Limouzy, and Nicolas Schivre. Output-linear time enumeration of potential maximal cliques. arXiv:2610.19850v3, 2026. Version of 20 October 2026. https://arxiv.org/abs/2610.19850v3.

[CS23] Florent Capelli and Yann Strozecki. Geometric amortization of enumeration algorithms. In *40th International Symposium on Theoretical Aspects of Computer Science (STACS 2023)*, volume 254 of *Leibniz International Proceedings in Informatics*, pages 18:1–18:22. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023. https://doi.org/10.4230/LIPIcs.STACS.2023.18.

[CS26] Florent Capelli and Yann Strozecki. From amortized to worst case delay in enumeration algorithms. *Computational Complexity*, 35, 2026. Article 6, published 20 July 2026. https://doi.org/10.1007/s00037-026-00287-w.

[GKTvdZ25] Alexander Grigoriev, Yasuaki Kobayashi, Hisao Tamaki, and Tom C. van der Zanden. A polynomial delay algorithm generating all potential maximal cliques in triconnected planar graphs. In *20th International Symposium on Parameterized and Exact Computation (IPEC 2025)*, volume 358 of *Leibniz International Proceedings in Informatics*, pages 21:1–21:17. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025. https://doi.org/10.4230/LIPIcs.IPEC.2025.21.

[Ken26] Batya Kenig. Enumerating small potential maximal cliques with fixed-parameter delay. arXiv:2610.24856v1, 2026. Version of 17 October 2026. https://arxiv.org/abs/2610.24856v1.

AI methodology statement

OpenAI Codex, based on GPT-6, was used substantially for mathematical research: developing and attacking the separator helper, the explicit neighborhood transfer, the recursive streaming algorithm and its timing induction; writing and running finite diagnostic code; consulting primary literature; and drafting and building this manuscript. A separate Codex context performed an internal mathematical and novelty review of the frozen argument and additional finite verification. Such review is neither external peer review nor formal verification, and no independent human verification of all mathematical claims or references is asserted here. The proof is presented in full with its source dependencies; the listed human author is responsible for assessing its accuracy, originality and suitability before any submission.